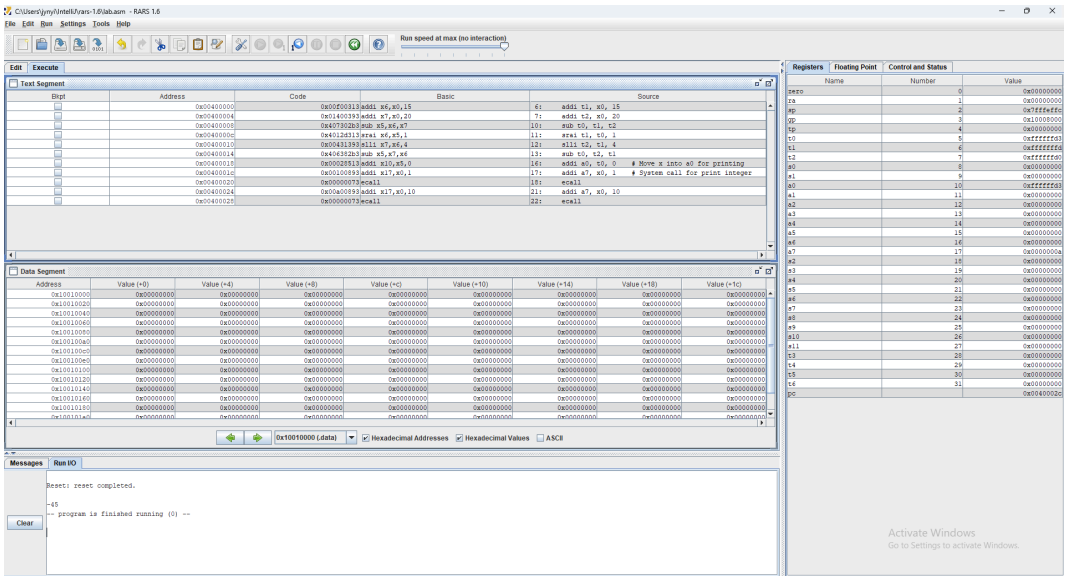
1. In the Hello World program, li instructions mean to load immediate, meaning we would be loading an integer rather than stored bits. To accomplish this using real instructions, one can use RISC instructions to perform moves.

   A7 is set multiple times due to needing different call values. Each value has a different call function; therefore, the value of a7 changes as different calls are needed.



2. I used srai because division requires arithmetic for rounding, which srli does not contain. If I were to use srli instead, the sign bits would not be preserved.

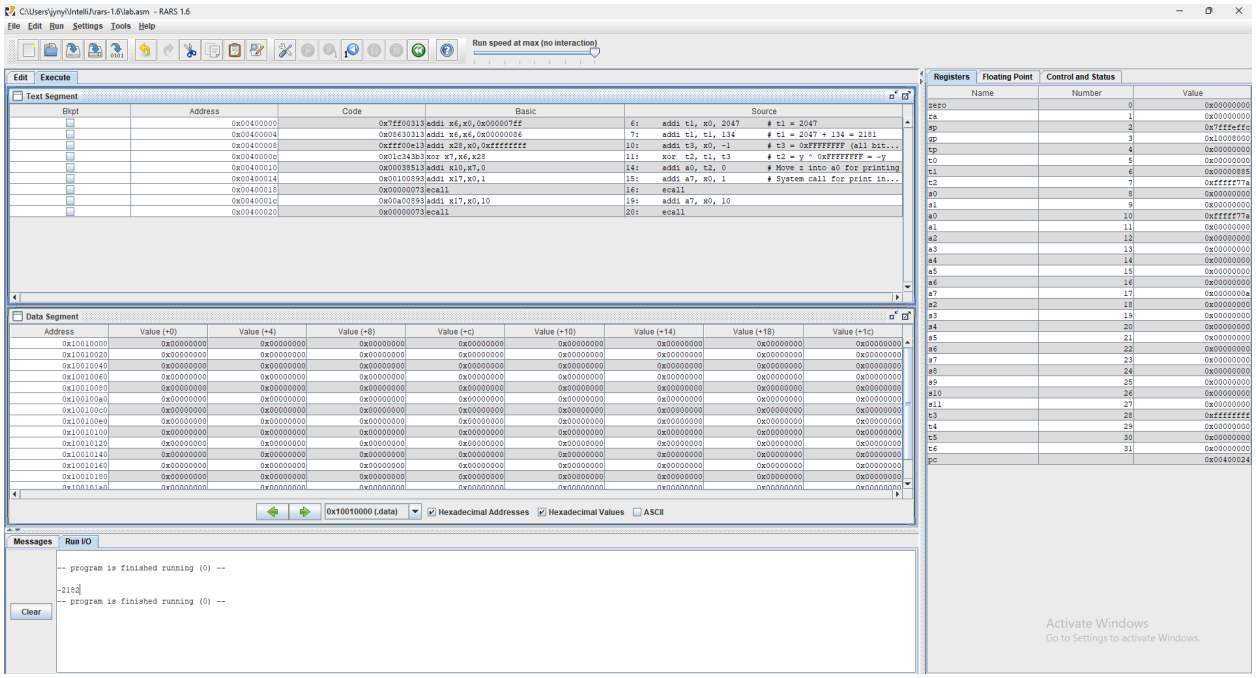
   To multiply by 17, I first multiplied by 16, then added the original value, making 17. This is more efficient, as shifting multiplies by powers of two, allowing a shortcut.
   X * 16 + X = X * 17

   The binary representation of 2181 is 100010000101 & not y would be 011101111010

   The result of z & y would be 0, as z and y are opposites; the AND op will cancel everything to 0. Similarly, the result of z | y would be 111111111111, as the OR op will merge everything to 1.

   Yes, it matches, as it follows the two's complement of flipping the bits and adding 1. Therefore, the decimal values also match that pattern of being one higher than the negative value.

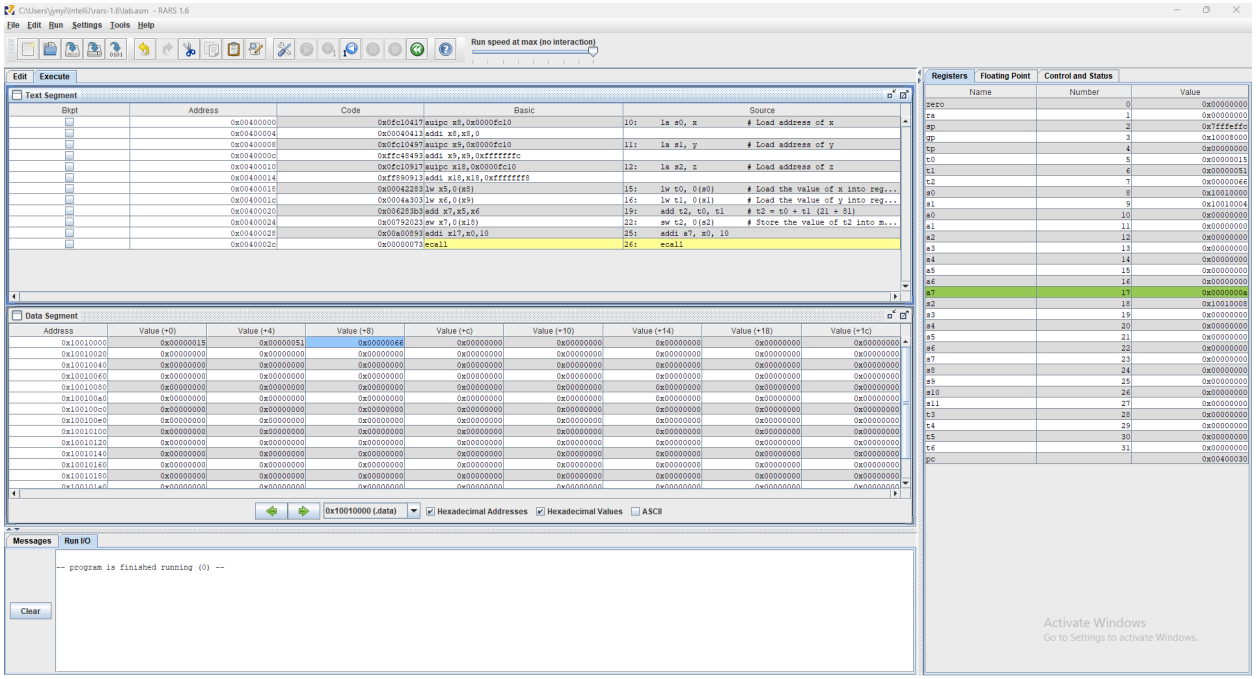
   Yes, you can use the XOR operator instead.



3. The value stored is now 102, stored using the sw function

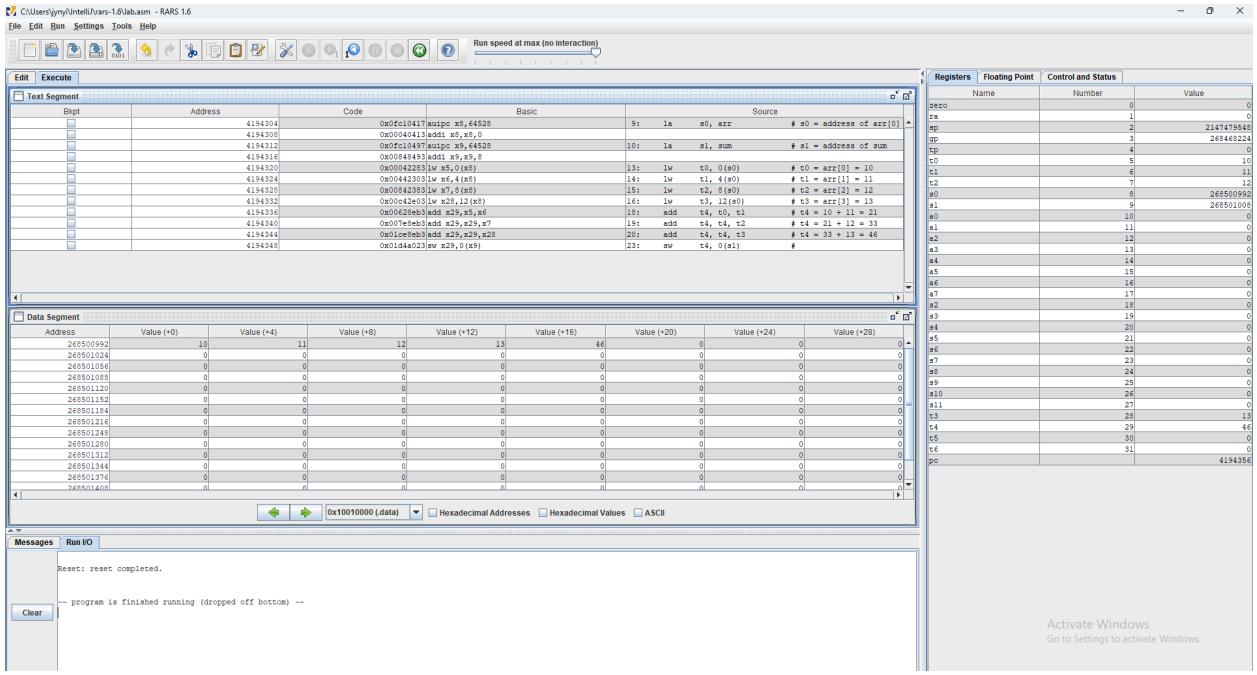
   It is necessary to use la to load the address first before storing as lw doesn't actually know the location/address. Therefore, la serves as a base address for lw to see.

   No, the memory would not change, as the sw function copies the value, not the actual register. Meaning, in this process, the register itself is independent of the memory.

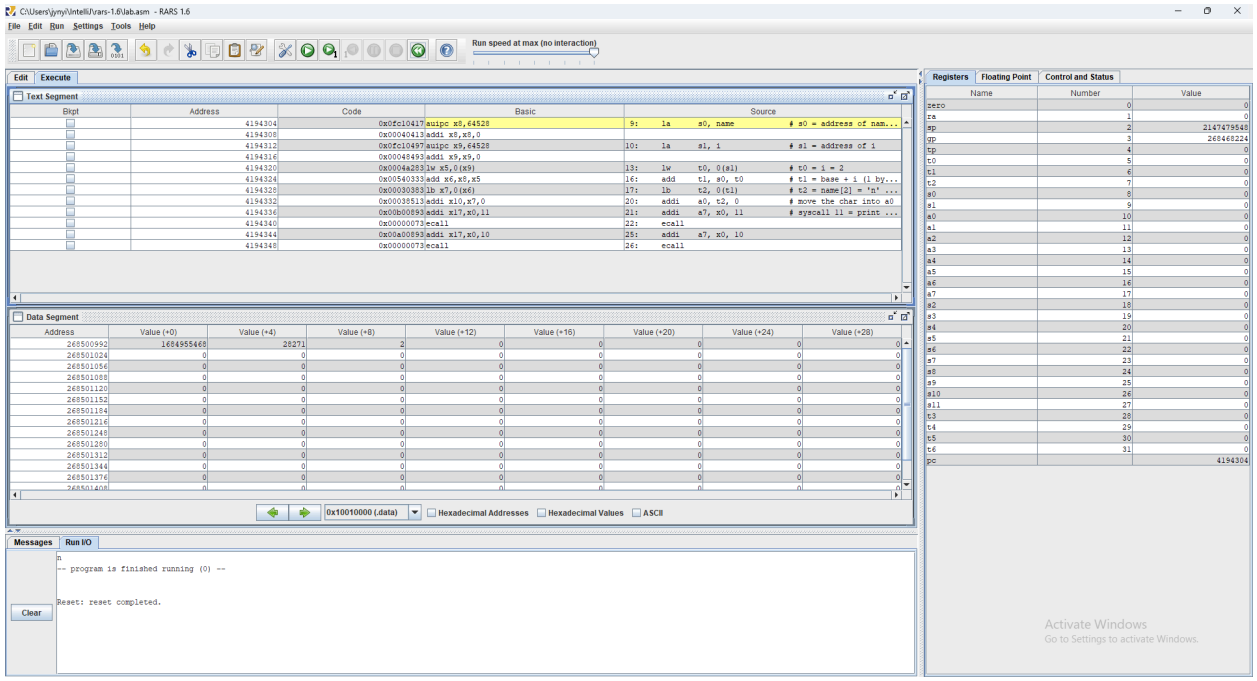


After the program completes, the value stored in sum (t4) is 46.

I calculated the memory offset by adding 4 per n offset. This is due to standard short integers being 4 bytes long or x32. If the array held long integers being 8 bytes or x64, it would instead be adding 8.



4.



I used lb to take the byte/char from the string. This functions differently from the normal lw, as it doesn't take the whole word, just the char.

The letter it printed was 'n', and that is correct. The reason it printed 'n' instead of the letter 'a' is due to the computer index starting from 0 rather than 1.

No, I didn't need to multiply, as each char is already 1 byte. If my program held integers instead, then it would be 4 bytes, meaning I would need to multiply by 4/shift by 2

I = 0 would load 'L', and setting i to be larger than the array or negative would return null or whatever was before the string. This is dangerous, as assembly performs no bounds checking, causing potential data leaks.